

EXP NO: 4A	SUPPORT VECTOR MACHINES (SVM)
-------------------	--------------------------------------

AIM

To build an **SVM model** for a binary classification task, tune its hyperparameters, and evaluate it using **accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC**.

ALGORITHM

1. Import the required libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn.
2. Load the **Breast Cancer Wisconsin dataset** from sklearn.datasets.
3. Split the dataset into **80% training** and **20% testing** data using a fixed random_state.
4. Standardize the features using StandardScaler.
5. Use SVC with an **RBF kernel** for classification.
6. Define hyperparameters C and gamma for tuning.
7. Perform **Grid Search with Cross-Validation** using GridSearchCV.
8. Select the best parameters based on cross-validation performance.
9. Train the final SVM model using the best parameters.
10. Predict the test data.
11. Calculate **accuracy, precision, recall, F1-score, and ROC-AUC**.
12. Display the **confusion matrix**.
13. Plot the **ROC curve**.
14. Report the best parameters and observations.

CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler
```

```

from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve
)

data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)

y = pd.Series(data.target, name="target")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

scaler = StandardScaler()

X_train_sc = scaler.fit_transform(X_train)

X_test_sc = scaler.transform(X_test)

# 5) Define model

svm = SVC(kernel='rbf', probability=True, random_state=42)

param_grid = {
    "C": [0.1, 1, 10, 100],
    "gamma": ["scale", 0.01, 0.001, 0.0001]
}

grid = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,
    scoring='f1',

```

```

cv=5,

n_jobs=-1,

verbose=0

)

grid.fit(X_train_sc, y_train)

print("Best Parameters from Grid Search:", grid.best_params_)

best_svm = grid.best_estimator_

best_svm.fit(X_train_sc, y_train)

y_pred = best_svm.predict(X_test_sc)

y_prob = best_svm.predict_proba(X_test_sc)[:, 1]

acc = accuracy_score(y_test, y_pred)

prec = precision_score(y_test, y_pred, zero_division=0)

rec = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)

auc = roc_auc_score(y_test, y_prob)

cm = confusion_matrix(y_test, y_pred)

print("\n=== SVM (RBF) — Test Metrics ===")

print(f"Accuracy : {acc:.4f}")

print(f"Precision: {prec:.4f}")

print(f"Recall   : {rec:.4f}")

print(f"F1-Score : {f1:.4f}")

print(f"ROC-AUC  : {auc:.4f}")

print("\nConfusion Matrix:\n", cm)

print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))

```

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.figure()

plt.plot(fpr, tpr, label=f"SVM (AUC = {auc:.3f})")

plt.plot([0, 1], [0, 1], linestyle="--", color="gray")

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve — SVM (RBF)")

plt.legend()

plt.grid(True)

plt.show()
```

OUTPUT:

Best Parameters from Grid Search: {'C': 10, 'gamma': 0.01}

=== SVM (RBF) - Test Metrics ===

Accuracy : 0.9825
Precision: 0.9861
Recall : 0.9861
F1-Score : 0.9861
ROC-AUC : 0.9977

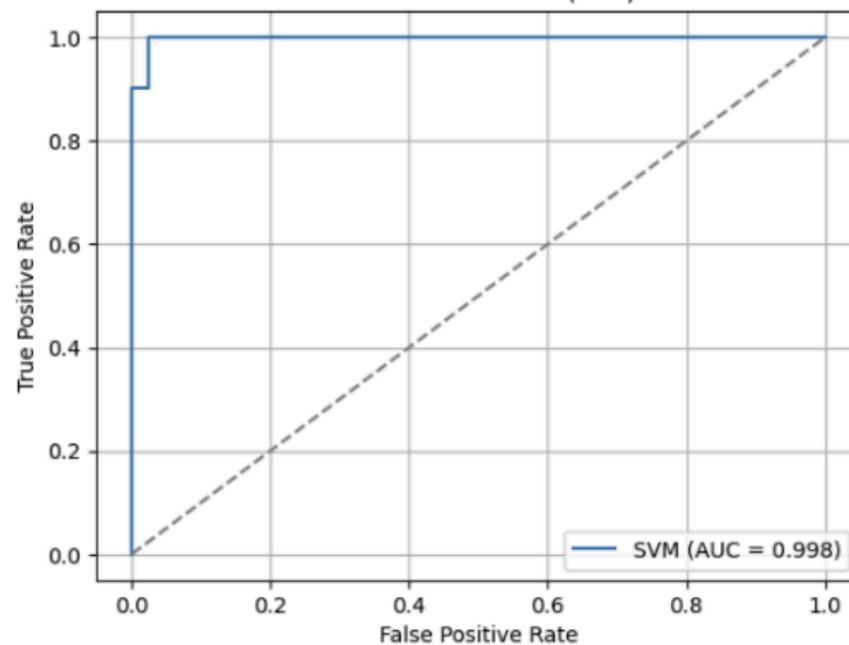
Confusion Matrix:

```
[[41  1]
 [ 1 71]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.98	0.98	42
1	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

ROC Curve — SVM (RBF)



EXP NO: 4B	ENSEMBLE METHODS: RANDOM FOREST
-------------------	--

AIM

To implement a Random Forest classifier for a classification task, tune key hyperparameters, evaluate its performance, and interpret feature importance.

ALGORITHM

1. Import the required libraries.
2. Load the Breast Cancer Wisconsin dataset, the same dataset used for SVM comparison.
3. Split the dataset into training and testing sets using stratification.
4. No feature scaling is required because Random Forest works with raw features.
5. Create a **RandomForestClassifier**.
6. Perform hyperparameter tuning using **GridSearchCV** for:
 - **n_estimators**
 - **max_depth**
 - **min_samples_split**
 - **min_samples_leaf**
7. Train the best Random Forest model using the training data.
8. Predict the test data.
9. Evaluate the model using:
 - Accuracy
 - Precision
 - Recall
 - F1-score
 - Confusion matrix
 - ROC-AUC
10. Extract feature importance values from the trained Random Forest.
11. Plot the top important features.
12. Report the best parameters, evaluation metrics, and important features.

CODE:

```
import numpy as np
```

```
import pandas as pd
```

```

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve
)

data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)

y = pd.Series(data.target, name="target")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

rf = RandomForestClassifier(random_state=42, n_jobs=-1)

param_grid = {
    "n_estimators": [100],
    "max_depth": [None, 10],
    "min_samples_split": [2],
    "min_samples_leaf": [1]
}

grid = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,

```

```
    scoring="f1",
    cv=3,
    n_jobs=-1,
    verbose=0
)

grid.fit(X_train, y_train)

print("Best Parameters (CV):", grid.best_params_)

best_rf = grid.best_estimator_

best_rf.fit(X_train, y_train)

y_pred = best_rf.predict(X_test)

y_prob = best_rf.predict_proba(X_test)[:, 1]

acc = accuracy_score(y_test, y_pred)

prec = precision_score(y_test, y_pred, zero_division=0)

rec = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)

auc = roc_auc_score(y_test, y_prob)

cm = confusion_matrix(y_test, y_pred)

print("\n=== Random Forest — Test Metrics ===")

print(f"Accuracy : {acc:.4f}")

print(f"Precision: {prec:.4f}")

print(f"Recall   : {rec:.4f}")

print(f"F1-Score : {f1:.4f}")

print(f"ROC-AUC  : {auc:.4f}")
```

```
print("\nConfusion Matrix:\n", cm)

print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))

importances = pd.Series(best_rf.feature_importances_, index=X.columns)

top10 = importances.sort_values(ascending=False).head(10)


plt.figure()

top10[::-1].plot(kind="barh")

plt.xlabel("Importance")

plt.title("Top 10 Feature Importances — Random Forest")

plt.grid(axis="x", alpha=0.3)

plt.show()

fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.figure()

plt.plot(fpr, tpr, label=f"Random Forest (AUC = {auc:.3f})")

plt.plot([0, 1], [0, 1], linestyle="--", color="gray")

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve — Random Forest")

plt.legend()

plt.grid(True)

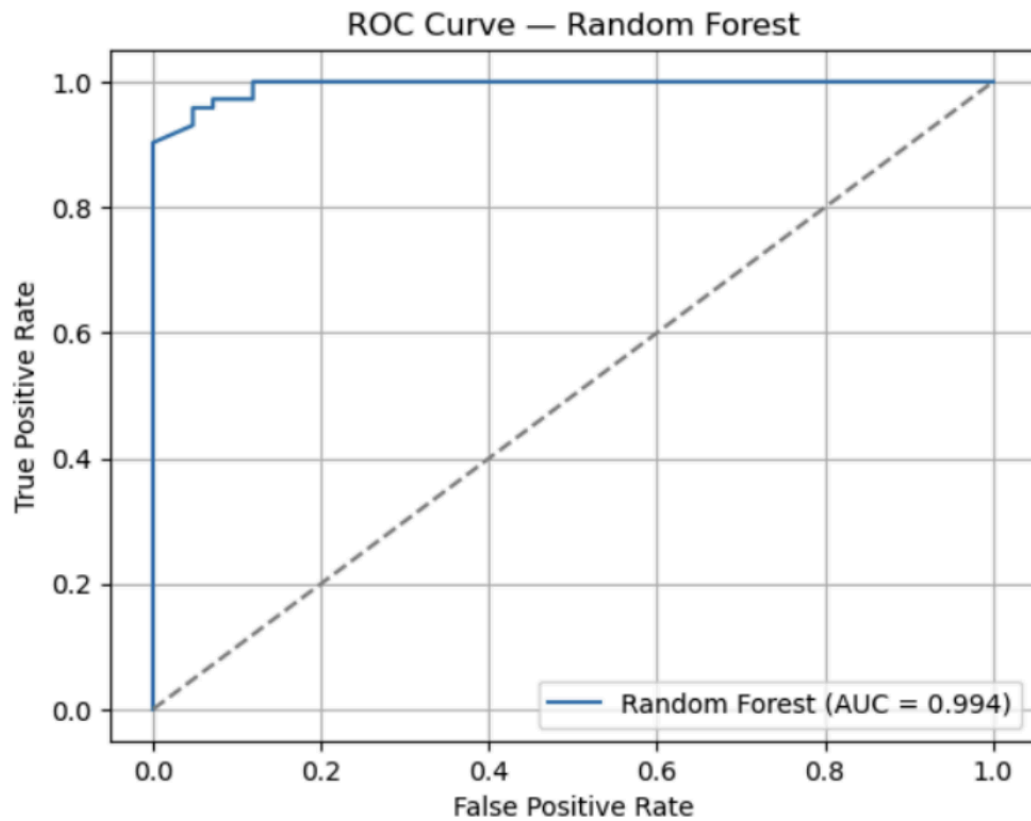
plt.show()
```

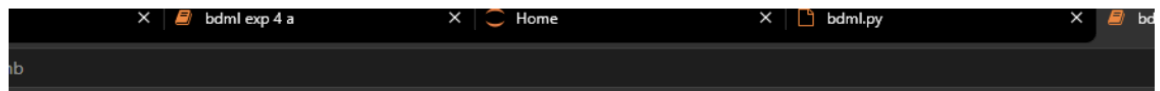
OUTPUT:

pyter bdml 4 a Last Checkpoint: 14 minutes ago

dit View Run Kernel Settings Help

✂ 📄 📋 ▶ ■ ↺ ▶▶ Code ▾ JupyterLab 🛡 Python (Anaconda3) ○ ☰





jupyter bdml 4 a Last Checkpoint: 14 minutes ago

Edit View Run Kernel Settings Help

+ Code

Best Parameters (CV): {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}

=== Random Forest - Test Metrics ===

Accuracy : 0.9561
Precision: 0.9589
Recall : 0.9722
F1-Score : 0.9655
ROC-AUC : 0.9937

Confusion Matrix:

```
[[39  3]
 [ 2 70]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.93	0.94	42
1	0.96	0.97	0.97	72
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

